

Reviewer Pack — Minimal Rank of Noncrossed Product K-Theory Bounds DPLL Tree...

Entry 1465de135f3f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 00:51:50 UTC

Minimal Rank of Noncrossed Product K-Theory Bounds DPLL Tree Depth

Entry ID: 1465de135f3f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 00:51:50 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Noncrossed Products) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

[‘For a given k-CNF formula, the minimal rank of the noncrossed product K-theory associated with its graphical realization is upper bounded by a function of the size n of the formula such that $E[\text{rank}(K)] \leq c * \log(n)$ ’, ‘where K is the K-theory of the noncrossed product and c is an absolute constant.’]

Rationale (proposer’s reasoning):

[‘The minimal rank of noncrossed product K-theory could provide a new perspective on the structure of DPLL search trees, potentially revealing hidden complexities that are not captured by current measures.’, ‘This approach has not been widely explored in complexity theory and offers a novel way to relate algebraic structures to computational problems.’]

Taxonomy category: NoncommutativeGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fc26c1bb8922a1c3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the empirical mean of the minimal ranks across 30 random seeds is less than or equal to $c * \log(n)$ for all $n \leq 40$, where c is an absolute constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        variables = set(range(1, n + 1))
        clauses = []
        for _ in range(k):
            clause = random.sample(variables, 2)
            clauses.append(clause)
        return clauses

    def graphical_realization(clauses):
        n = len(clauses)
        graph = {i: [] for i in range(n)}
        for i in range(n):
            for j in range(i + 1, n):
                if set(clauses[i]) & set(clauses[j]):
                    graph[i].append(j)
                    graph[j].append(i)
        return graph

    def noncrossed_product_k_theory(graph):
        n = len(graph)
        rank = 0
        for i in range(n):
            neighbors = graph[i]
            if not neighbors:
                continue
            matrix = [[0] * (n + 1) for _ in range(n + 1)]
            for j in neighbors:
                matrix[i][j] = 1
                matrix[j][i] = 1
            matrix[n][i] = -1
            matrix[i][n] = -1
```

```

        rank += gaussian_elimination(matrix)
    return rank

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i + 1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            else:
                return 0
        pivot = matrix[i][i]
        for j in range(n + 1):
            matrix[i][j] /= pivot
        for j in range(n):
            if i != j:
                factor = matrix[j][i]
                for k in range(n + 1):
                    matrix[j][k] -= factor * matrix[i][k]
    return sum(1 for row in matrix if any(row))

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        k = random.randint(1, min(n * (n - 1) // 2, 10))
        clauses = generate_k_cnf(n, k)
        graph = graphical_realization(clauses)
        rank = noncrossed_product_k_theory(graph)
        total_rank += rank
        instances_tested += 1

mean_rank = total_rank / instances_tested
conjecture_holds = mean_rank <= 2 * math.log(n) for n in n_values

return {
    "metric_name": "mean_rank",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": all(conjecture_holds),
    "counterexample": "" if all(conjecture_holds) else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

```

```

mean_rank = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15

```

```

Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
Testing n=5
Testing n=15
Testing n=25
Testing n=35
TRIAL: {'metric_name': 'min_rank', 'metric_value': 1.0, 'instances_tested': 360, 'conjecture_holds':
RESULT: SUPPORTED mean=0.00277777777777778 std=0 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only includes $n \leq 15$ and does not scale with n , which may not be representative of the conjecture's validity for larger values of n . Additionally, the metric saturation is a concern since there is no evidence that the bound $c * \log(n)$ cannot be trivially achieved by all instances.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test only includes $n \leq 15$ and does not scale with n , which may not be representative of the conjecture's validity for larger values of n . Additionally, there is no evidence provided that the bound $c * \log(n)$ cannot be trivially achieved by all instances. | next: Expand the range of n to include values up to 40 and ensure that the empirical mean of the minimal ranks across multiple seeds is consistently less than or equal to $c * \log(n)$.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10662
2	preregistration	ollama_remote	glm4:latest	0	0	5350
3	novelty	ollama_remote	glm4:latest	0	0	4824
4	novelty	ollama_remote	glm4:latest	0	0	5269
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33512
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8613
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11468
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12180
9	critic	ollama_remote	glm4:latest	0	0	13486
10	judge	ollama_remote	glm4:latest	0	0	6764

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112128 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/1465de135f3f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1465de135f3f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1465de135f3f.tar.gz` (if generated)